

PROJECT EXPLAINER

NADRA Guide RAG Assistant

What the project does, the technology behind it, and how the repository is organized.

ONE-SENTENCE SUMMARY

A multilingual web and command-line assistant that retrieves relevant passages from official NADRA documents and asks a Groq-hosted language model to produce clear, source-aware answers about identity services.

COVERS

CNIC

NICOP

POC

CRC

FRC

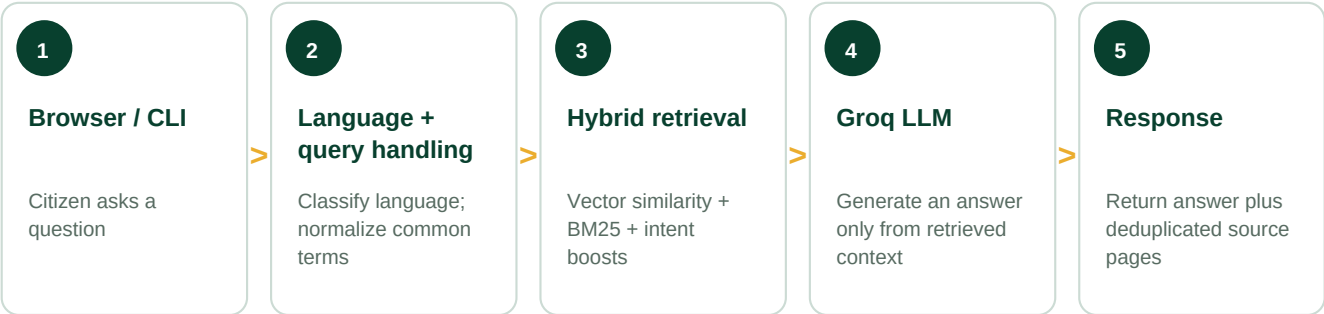
1. What does this project do?

It turns a collection of NADRA reference documents into a conversational guidance system.

USER-FACING BEHAVIOR

- Answers questions about CNIC, NICOP, POC, CRC and FRC: eligibility, required documents, fees, timelines, procedures and office locations.
- Understands English, Urdu script and Roman Urdu, then responds in the user's language style.
- Carries up to 12 recent chat messages so follow-up questions and translation requests can use conversation context.
- Shows the source document and page behind an answer when the generated response actually uses them.
- Refuses out-of-scope or unsupported requests instead of intentionally filling gaps with general knowledge.

HOW ONE QUESTION IS ANSWERED



IMPORTANT BOUNDARY

This is an information assistant, not an official transaction system. It does not issue cards, access citizen records, verify identities or replace NADRA. Answer quality depends on the documents loaded into the local knowledge base and availability of the Groq API.

2. What tech stack does it use?

A Python RAG backend, a lightweight browser frontend and a locally persisted retrieval index.

LAYER	TECHNOLOGY	ROLE
Backend API	Python, FastAPI, Uvicorn, Pydantic	Validates chat requests, exposes /api/chat and /api/health, and serves the frontend.
RAG orchestration	LangChain Core, langchain-groq	Builds prompts and calls Groq's Llama 3.3 70B Versatile model at temperature 0.
Semantic search	FastEmbed, ONNX Runtime, multilingual-e5-small	Creates 384-dimensional multilingual embeddings locally on CPU.
Keyword search	rank-bm25	Finds exact terms such as service acronyms, fee labels and form fields.
Vector storage	NumPy + JSON files	Stores embeddings.npy, documents.json and manifest.json under data/vector_store.
Document processing	PyMuPDF, LangChain text splitters	Extracts PDF pages, cleans text and splits it into 1,000-character chunks with 150-character overlap.
Frontend	HTML, CSS, vanilla JavaScript	Provides the responsive chat interface, local conversation state and source display.
Frontend tooling	Vite 7 (development dependency)	Runs a dev server and production build; Node.js is optional for the normal FastAPI run.
Configuration	python-dotenv + .env	Loads GROQ_API_KEY; secrets remain outside committed source.

Deployment profile: single FastAPI process for the standard setup; embeddings and the vector index are local, while answer generation is remote through Groq.

3. How is the project structured?

The repository separates web delivery, RAG logic, knowledge-base artifacts, source documents and evaluation utilities.

NADRA-RAG-main/

```
|-- app.py                FastAPI entry point and static-file server
|-- src/
|   |-- rag_pipeline.py    language policy, prompting, retrieval and answer flow
|   |-- kb.py              embeddings, local vector store and hybrid retriever
|   |-- ingest.py          PDF/text loading, cleaning, chunking and indexing
|   |-- chat_cli.py        terminal chat interface
|   |-- transcribe_forms.py scanned-form transcription helper
|-- frontend/
|   |-- index.html         interface markup
|   |-- styles.css         responsive presentation
|   |-- main.js            chat state, API calls and source rendering
|   |-- package.json       optional Vite development/build setup
|-- pdfs_data/            source PDFs and transcribed text sidecars
|-- data/
|   |-- vector_store/      active documents, vectors and model manifest
|   |-- fastembed_cache/   cached ONNX embedding model
|-- evaluation/           retrieval, answer, language and citation checks
|-- requirements.txt       Python dependencies
|-- .env.example          API-key template
|-- README.md             setup, API and data instructions
```

RESPONSIBILITIES AT A GLANCE

Web layer

`app.py` + `frontend/`

Receives the question, preserves recent chat context and renders the answer.

Reasoning layer

`src/rag_pipeline.py`

Applies language and safety rules, prepares context and calls the LLM.

Retrieval layer

`src/kb.py`

Combines semantic and lexical rankings using reciprocal rank fusion plus intent boosts.

Data pipeline

`src/ingest.py` + `pdfs_data/`

Converts reference material into the searchable local index.

How the pieces work together

There are two distinct workflows: rebuilding the knowledge base and serving questions.

A. Build or refresh the knowledge base

- 1 Place official PDF files in `pdfs_data/`; keep authoritative text sidecars in `pdfs_data/transcribed/`.
- 2 Run `python src/ingest.py`.
- 3 PyMuPDF extracts pages; recurring boilerplate and near-empty content are removed.
- 4 Text is split into overlapping chunks and embedded locally with `multilingual-e5-small`.
- 5 The resulting JSON metadata and NumPy vectors are persisted in `data/vector_store/`.

B. Run and use the assistant

- 1 Start `uvicorn app:app --reload` and open `http://127.0.0.1:8000`.
- 2 FastAPI warms the local retriever, then serves the browser interface and REST API.
- 3 `POST /api/chat` receives a question plus recent history and runs the RAG pipeline in a worker thread.
- 4 The pipeline retrieves five chunks by default, asks Groq for a grounded answer, checks language rules and returns sources.
- 5 The same `answer_question` function is also available through `src/chat_cli.py`.

USEFUL ENTRY POINTS

Web app

`app.py`

Core Q&A

`src/rag_pipeline.py`

Retrieval/index

`src/kb.py`

Re-index command

`src/ingest.py`

Automated checks

`evaluation/evaluate.py`

Bottom line

The project is deliberately small and local-first: documents and embeddings stay on disk, retrieval runs on the user's machine, and only the selected context plus the conversation prompt is sent to Groq for answer generation.